Deductive LLVM verification in Isabelle

Research Topics

Thomas Kas
t.s.kas@student.utwente.nl

2026-03-30

Contents

1 Introduction . . . 3
1.1 Motivating Example . . . 3
1.2 Verification Methods . . . 3
1.3 Verification Targets . . . 4
1.4 Verification Back-end . . . 4
1.5 Research Goals . . . 4
1.6 Proposal Note . . . 5
2 Background . . . 5
2.1 Deductive Verification . . . 6
2.2 LLVM and LLVM-IR . . . 9
2.3 Isabelle . . . 12
3 Preliminary Results (WIP) . . . 15
3.1 LLVM-IR in Isabelle . . . 15
3.2 Verification Infrastructure . . . 17
3.3 Verification Tools . . . 17
4 Related Work (WIP) . . . 18
5 Planning . . . 18
5.1 Risk Mitigation . . . 19
Bibliography . . . 20

1 Introduction

As software is used in more and more safety-critical areas, the importance of its verification grows rapidly. It is used to control modes of transportation like cars and planes, critical infrastructure like power plants and electrical grids, and many other areas. If such software fails, the consequences are great, which means verifying that such pieces of software work as intended has become crucial. The goal of this thesis is to create a verification tool for most software regardless of source language or target architecture by strategically choosing an optimal verification method and target combination.

1.1 Motivating Example

A simple example of a C program to be verified is shown in Listing 1. It consists of only the function `mult`, which takes two parameters `a` and `b`, and should result in an integer of these inputs multiplied together. Its implementation is not quite straight-forward as it does not use any multiplication instruction, instead relying on a for-loop with repeated additions. As such, verification of even such a relatively simple program requires in-depth understanding of the semantics of C.

```c
int mult(int a, int b) {
    int result = 0;
    for (int i = 0; i < b; i++) {
        result += a;
    }
    return result;
}
```

Listing 1: C `mult` Function Implementation

1.2 Verification Methods

This verification can be done through various different methods. One such method is model checking, where a model of the system the software acts upon is created and used to verify the software functions correctly in some or all possible states of the model. Another method is dynamic analysis, where (part of) the software is run and its results or execution states are checked against some expected outcome or allowed bounds, commonly implemented in unit or acceptance tests. In contrast to dynamic analysis, static analysis techniques can be used to identify issues without running the software. The issues it can identify usually consist of common programming mistakes and is limited in its scope, lacking the strength to verify an implementation matches the expected behavior. Finally, there is deductive verification, which mathematically verifies correctness using logical reasoning based on pre- and post-conditions. It is using this last method, deductive verification, that a new verification tool will be created in this thesis, as its formal mathematical basis allows for the most rigorous verification of these approaches.

1.3 Verification Targets

Deductive verification can be done at various stages in compilation of software. It might be done with the source code, with fully compiled binaries, or with some intermediary between these stages. Performing verification on source code comes with the advantage that it is relatively easy to integrate the verification into the program, for example with in-line verification instructions, but this comes at the cost of the verification being tied to the specific programming language the software was written in. Targeting a lower level like machine instructions gets around this issue and is the closest to verifying the actual execution of a program, as compilation might introduce artifacts not present in source code, but means that only a single target is verified, tying the verification to a specific architecture. In both cases, changing the language or architecture the verification is done with means porting the verification to a different target as well, introducing more overhead for such tasks.

This thesis aims to create a tool for deductive verification at a level between these two, namely LLVM-IR. This is the intermediate representation of LLVM, a compilation framework that has separate front-ends and back-ends for different source languages and target architectures. It allows language-engineers to only target a single compilation representation, LLVM-IR, while supporting many different target architectures, including x86, ARM, and RISC-V. Furthermore, it allows developers working on new hardware targets to implement a single back-end for LLVM-IR, automatically enabling support for all languages that compile to it. For our proposed verification tool this means that, once functional, it will support software written in a wide range of source languages and targeting many target architectures.

1.4 Verification Back-end

Finally, the chosen formal verification back-end for this thesis is Isabelle/HOL. This is an interactive theorem prover that allows users to specify theorems using higher-order logic and mathematically prove them to be correct. Whenever a new theorem is created, it gives the user proof (sub-)goals that have to be proven for the theorem to be accepted. Different tools and solvers are provided to the user which aid in simplifying, breaking up, and eventually proving these goals. Furthermore, it is possible to define custom proof methods for specific tasks using the Eisbach infrastructure. Isabelle is also a functional programming language, allowing users to create their own datatypes and functions, and letting them use these in their lemmas. All of these features combined make Isabelle well-suited to defining the structure and semantics of LLVM-IR and creating verification tools that are straight-forward to use on top of that.

1.5 Research Goals

With that, the main research goal of this thesis is:

RG - *Creating a deductive verification tool for partial correctness of LLVM-IR programs within Isabelle/HOL.*

This can be achieved through the following sub-goals, producing a minimum viable product:
`SG1` - *Defining the semantics of a targeted subset of LLVM-IR in Isabelle.*
`SG2` - *Implementing deductive verification infrastructure that works with these semantics.*
`SG3` - *Creating tools to efficiently and effectively verify LLVM-IR programs.*
`SG4` - *Using these tools on some program to show its efficacy.*

Once these goals are achieved and an MVP has been produced it can be extended with multiple extension goals, such as:
`EG1` - *Extending the defined subset of LLVM-IR to support more of its features, instructions, and intrinsics.*
`EG2` - *Creating another tool to port LLVM-IR output into the verifier and vice versa, preferably using Isabelle to prove it can losslessly port LLVM-IR programs.*
`EG3` - *Improving the backend using advanced verification principles, such as separation logic, to make verification easier.*
`EG4` - *Extending the verifier to prove total correctness rather than only partial correctness.*
These extension goals are not strictly necessary to achieve the main research goal, but do enhance the functionality of the created tool.

1.6 Proposal Note

Work has already begun implementing the tool described in this proposal, following the philosophy that practical experience is a great way to better understand choices made in existing literature. As such, in addition to giving background information on the techniques and technologies this thesis will incorporate, this proposal also describes the work that has already been done towards the goals of the thesis. This makes the proposal slightly larger than other similar research proposals, as it covers this work in some detail.

Specifically, Section 2 gives background information on techniques and technologies to be used for this thesis, Section 3 covers preliminary results from the work that has already been done, Section 4 covers existing literature in this space to take inspiration from, and finally Section 5 gives a rough planning on how work will proceed from this point forward to complete the proposed tool, as well as some implementation risks and mitigation strategies should they arise.

2 Background

This section provides some background information on the different concepts and tools that lay at the foundation of this thesis. Specifically, Section 2.1 explains how deductive verification works and how it is done, then Section 2.2 explores how LLVM-IR is structured and how it works semantically, and finally Section 2.3 introduces the different features of Isabelle/HOL that will be used to achieve the different goals of this thesis.

2.1 Deductive Verification

Deductive verification is a method to mathematically verify the correctness of programs based in pre- and post-conditions. There exist many different deductive verifiers for different languages and toolchains, based on mostly the same principles.

2.1.1 Hoare Triples

At the core of deductive verification lie pre-conditions, commands, and post-conditions. The combination of the three, known as Hoare triples, allows one to reason about programs. The pre-conditions, P, represent some conditions that have to hold for the command, c, to execute properly, while the post-conditions, Q, represent the intended result of the command.

```c
/*@
requires a * b < 2^31
requires b >= 0;
ensures \result == a * b;
*/
int mult(int a, int b);
```

Listing 2: C `mult` Function Annotated with ACSL Specifications

Consider Listing 2. It contains a specification for the function `mult`, written in ACSL, a specification language used in deductive verification of C programs using Frama-C. This specifies two pre-conditions and one post-condition for the function, which represents a command. Specifically, the pre-conditions state that the inputs `a` and `b` multiplied should not exceed the signed 32-bit integer limit and `b` should be non-negative for the function to execute properly. If this holds, then the post-condition states that the result will be the inputs `a` and `b` multiplied.

To prove that a post-condition follows from the pre-conditions and the command, the semantic meaning of the command has to be described formally. Consider again the implementation of the `mult` function in Listing 1. To prove anything about this function, the prover needs to know what exact effects the instructions have on the execution state: how values of the `int` type behave, how assigning variables behaves, how different operations such as ++ and += interact these values, and how control-flow is impacted by the `for`-loop and `return` keyword.

Once the prover knows the semantics of different instructions, it can assume the pre-conditions hold for the inputs and execute the instructions of a program symbolically, keeping track of how execution state and flow have been impacted thus far. When a return instruction is reached, the post-condition can be checked on the returned value and reached execution state. If it holds for all possible executions, then the program satisfies the specification and is verified deductively. Note that this only proves partial correctness:

it might be possible for the program not to terminate, but if it does, then the post-condition holds.

2.1.2 Floyd's Method

With that, it is crucial to know the control-flow of all possible executions of a program, as it does not suffice to show correctness for only some executions. For this purpose, Floyd's method can be applied. This involves creating a control-flow graph of the program and creating specifications for all branch-points in said graph. What results is a number of instruction blocks, each annotated with some pre-conditions, and a graph containing the flow between these blocks.

Verification of programs using the annotated instruction blocks and their flow becomes straight-forward. Starting with the first block, its pre-conditions are assumed to hold, and its instructions are executed symbolically. From there, the resulting execution state has to be shown to satisfy the pre-conditions of all possible next instruction blocks. This has to be repeated for all of the annotated blocks. Finally, whenever a block ends by returning, the post-condition of the program has to be shown to hold from the pre-condition and execution of the block. When this is done, the program has been verified to be (partially) correct.

Consider Listing 3. Since the for-loop represents a branch-point in control-flow, some pre-conditions have to be specified for these instruction blocks. For this purpose, loop invariants have been added to the for-loop in the `mult` function. The pre-conditions of the function become the pre-conditions of the initial instruction block, while the loop invariants become the pre-conditions for the loop and final instruction blocks. This is visualized in Figure 1.

```c
/*@
requires a * b < 2^31;
requires b >= 0;
ensures \result == a * b;
*/
int mult(int a, int b) {
    int result = 0;
    /*@
    loop invariant 0 <= i <= b;
    loop invariant result == a * i;
    */
    for (int i = 0; i < b; i++) {
        result += a;
    }
    return result;
}
```

Listing 3: C `mult` Function Implementation Annotated with ACSL Specifications

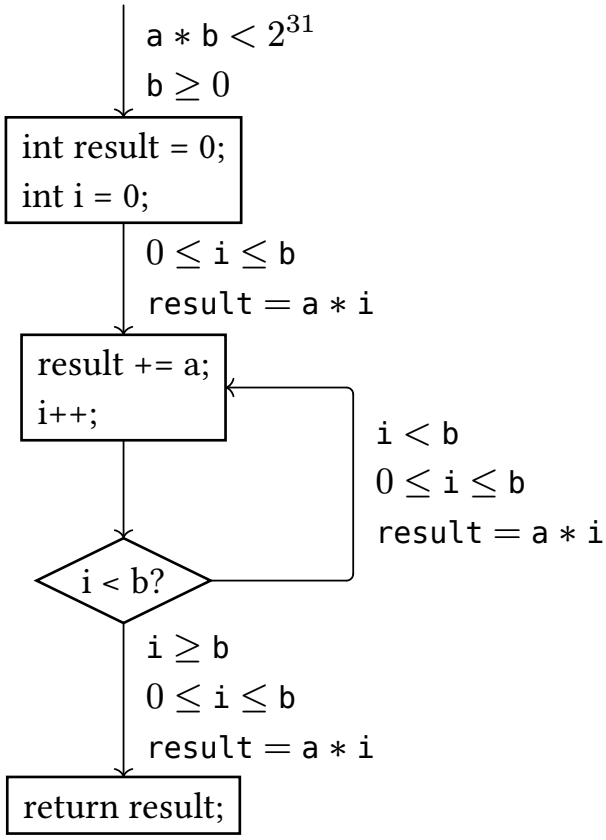

Figure 1: Annotated `mult` Control Flow Graph

With that, the goal of the deductive verification infrastructure is to:

- allow users to specify pre-conditions per block of instructions with linear control-flow and a general post-condition,

- allow for the symbolic execution of instruction blocks with some assumed conditions, obtaining the execution state,
- allow proving that the pre-conditions of all consequent instruction blocks hold for some obtained execution state, or the post-condition in case the block returned some value.

2.2 LLVM and LLVM-IR

LLVM is a toolkit for the construction of compilers, optimizers, and run-time environments. At its core is its own intermediate representation, LLVM-IR, a low-level programming language similar to assembly. It can be used to develop front-ends for any programming languages, and back-ends for any target architectures. Currently supports compiling many source languages, including C, C++, Haskell, and Rust, to a wide range of target architectures, including x86-64, ARM, and RISC-V.

2.2.1 LLVM-IR Structure

LLVM-IR is composed at the highest level of different modules, which themselves are made up of functions, global variables, and symbol table entries. The functions consist of an outer definition, including its parameters and return type, and inner basic blocks. These basic blocks are made up of a block label (which are local identifiers), followed by zero or more phi instructions, then some regular instructions, and finally one terminal instruction. Control-flow between basic blocks are determined only by these terminal instructions, as they either specify a local identifier to branch to, or return from the function. The other instructions can only affect the outer control-flow by, for example, calling another function, but not the inner control-flow of the function being executed.

LLVM-IR has two different ways of storing data: in registers, and in memory. Its registers are function-local single-static-assignment values, accessible via identifiers. In other words, they can only be assigned once statically, and their values only remain accessible within the function they are defined in. Their values can however be overwritten when the instruction that assigns them is executed again. Meanwhile, its memory is accessible via specific instructions such as `load` and `store`. These instructions are given the memory addresses to operate on using registers.

Identifiers start with `%` for these local identifiers, and `@` for global identifiers (used for functions and global variables). They have to be unique, either at a local level for local identifiers, or globally for global identifiers. Whenever a value is used as an operand for an instruction, it can either be specified as the identifier of a register, or a constant value.

As an example of a function in LLVM-IR, Listing 4 shows the `mult` function from Listing 1 compiled to LLVM-IR. Its definition specifies its return type is `i32`, and has two `i32` parameters, accessible in registers `%a` and `%b`. It contains 5 different basic blocks: `entry`, `for.cond`, `for.body`, `for.inc`, and `for.end`. None of these blocks contain phi instructions, and are made up of the instructions `alloca`, `store`, `load`, `icmp`, and `add`. They contain the terminal instructions `br` (both the direct `label` and conditional `i1` branch variants) and `ret`. The syntax of these instructions is given in Listing 5.

```llvm
define dso_local noundef i32 @mult(int, int)(i32 noundef %a, i32 noundef %b) {
entry:
  %a.addr = alloca i32, align 4
  %b.addr = alloca i32, align 4
  %result = alloca i32, align 4
  %i = alloca i32, align 4
  store i32 %a, ptr %a.addr, align 4
  store i32 %b, ptr %b.addr, align 4
  store i32 0, ptr %result, align 4
  store i32 0, ptr %i, align 4
  br label %for.cond

for.cond:
  %0 = load i32, ptr %i, align 4
  %1 = load i32, ptr %b.addr, align 4
  %cmp = icmp slt i32 %0, %1
  br i1 %cmp, label %for.body, label %for.end

for.body:
  %2 = load i32, ptr %a.addr, align 4
  %3 = load i32, ptr %result, align 4
  %add = add nsw i32 %3, %2
  store i32 %add, ptr %result, align 4
  br label %for.inc

for.inc:
  %4 = load i32, ptr %i, align 4
  %inc = add nsw i32 %4, 1
  store i32 %inc, ptr %i, align 4
  br label %for.cond

for.end:
  %5 = load i32, ptr %result, align 4
  ret i32 %5
}
```

Listing 4: `mult` Function compiled to LLVM-IR

```llvm
<result> = phi [fast-math-flags] <ty> [ <val0>, <label0>], ...

<result> = alloca [inalloca] <type> [, <ty> <NumElements>] [, align <alignment>] [, addrspace(<num>)]
store [volatile] <ty> <value>, ptr <pointer>[, align <alignment>]
<result> = load [volatile] <ty>, ptr <pointer>[, align <alignment>]
<result> = add [nuw] [nsw] <ty> <op1>, <op2>
<result> = icmp [samesign] <cond> <ty> <op1>, <op2>

br i1 <cond>, label <iftrue>, label <iffalse>
br label <dest>

ret <type> <value>
ret void
```

Listing 5: Syntax of LLVM-IR Instructions

2.2.2 LLVM-IR Semantics

When executing functions, its first instruction block (which is the only one where a label is not mandatory) is executed first. From there, the terminal instructions of blocks dictate whether another block is executed, or the function returns. Within blocks, instructions are executed in-order. As this thesis will only encapsulate a sub-set of LLVM-IR instructions, only the semantics of those included will be shown here, which are: `phi`, `alloca`, `store`, `load`, `add`, `icmp`, `br`, and `ret`. As the scope of the thesis increases to cover more instructions, this section will grow to also explain their semantics.

The fact that LLVM-IR functions are already subdivided into basic blocks with straightforward control-flow makes it ideal for graph-based deductive verification. Each basic block can only lead to one other predetermined basic block, or to one of two based on a boolean value, so constructing a control-flow graph is simple. With that, the biggest challenge for the design of the verifier is how to annotate basic blocks with pre-conditions and how to symbolically execute instructions from there, keeping track to register and memory state modifications.

2.2.2.1 Phi Instructions

The `phi` instructions always have to appear at the start of an instruction block. They assign some value from the list `[ <val0>, <label0>], ...` to the register `<result>`, specified by the pair corresponding to the predecessor block that branched to this block. As such, it allows for a conditional register assignment which is a valid SSA-value.

2.2.2.2 Regular Instructions

`alloca` instructions allocates memory on the stack frame of the current function, which is released automatically when the function returns. They yield an address space which is stored in the register `<result>`.

`store` instructions store a value (from a register or a constant value) at some memory address from a pointer (e.g. an address space in a register, as return by `alloca`). It does not yield any value.

`load` instructions read a value at a memory address from a pointer. It yields the read value to be stored into the register `<result>`.

`add` instructions add together two values of the same type, `<op1>` and `<op2>` and yields the result to be stored into `<result>`. Depending on whether `nuw`, no unsigned wrap, or `nsw`, no signed wrap, were specified, and either an unsigned or signed overflow occurred respectively, a poison value is produced. Such a poison value propagates whenever it is used in any computations, and leads to undefined behavior when it is used for certain instructions, for example those that affect control-flow and memory addressing.

`icmp` instructions compare two integer values according to the specified comparison operand `<cond>`. This is one of `eq` for equal, `ne` for not equal, `ugt` for unsigned greater than, `uge` for unsigned greater or equal, `ult` for unsigned less than, and `ule` for unsigned less or equal comparisons, or `sgt`, `sge`, `slt`, and `sle` for their signed counterparts. If `samesign` was specified then the operation produces a poison value if the operands are not of the same sign. It yields a single-bit integer (a boolean) for the comparison to be stored into `<result>`.

2.2.2.3 Terminal Instructions

`br label` instructions branch to the basic block with the specified label unconditionally, while `br i1` branches to the block with the first label if the value `<cond>` is true, or the block with the second label if it is false. This has undefined behavior is `<cond>` is a poison value.

`ret` returns from a function with the specified `<value>`, or with no value if the type is `void`.

2.3 Isabelle

Isabelle/HOL (referred to as just Isabelle from here) is an interactive theorem prover with higher-order logic features, allowing users to create and prove correct theorems consisting of lemmas. These lemmas can contain several assumptions, and define a single conclusion from the assumptions. Whenever a new lemma is defined using the keyword `lemma`, Isabelle prompts the user to prove one or more proof goals. With provided tools, the user is able to simplify, break up, rewrite, and eventually prove these goals (assuming they are correct). Once all proof goals have been proven, the lemma is added as correct to a theorem.

Isabelle is also a functional, statically-typed language, allowing users to create functions that can then be used in lemmas. Functions can have several parameters of specific types, and always produce a result of a single type. They can use pattern matching over the

parameters to specify different function definitions depending on the input. Simple recursive functions created using the keyword `fun` are automatically proven to terminate, but some non-recursive functions created using the keyword `function` or `partial_function` require the user to prove they terminate. As an example, the `mult` function has been written in Isabelle as a recursive function over the second parameter in Listing 6. Here, the parameters are natural numbers, which are defined as zero, `0`, or recursively as the successor of another natural number, `Suc n`. If the second parameter is zero, then the result of the function is also zero, but if it is the successor of a natural number, `b`, the function recurses to `mult a b` (`a` being the first parameter) and adds `a` to that.

```isabelle
fun mult where
  "mult a 0 = 0"
| "mult a (Suc b) = a + mult a b"
```

Listing 6: Recursive Function `mult` in Isabelle

Proving that the `mult` function indeed multiplies the two inputs is possible by creating a lemma that states `mult a b` is equal to `a * b`, and proving the resulting proof goals. This has been done in Listing 7. The proof works as follows:

- applying induction over `b` using `apply (induction b)`, to get two new sub-goals:
  - the base case: `mult a 0 = a * 0`
  - the inductive step: $\bigwedge$`b. mult a b = a * b` $\Longrightarrow$ `mult a (Suc b) = a * (Suc b)`
- applying the simplifier to the base case using `apply simp`, which internally does the following to the sub-goal:
  - rewrite `mult a 0` to its definition: `0`,
  - rewrite `a * 0` to `0` using an internal definition of multiplication of natural numbers,
  - prove the resulting sub-goal `0 = 0` by reflection.
- applying the simplifier to the inductive step using `apply simp`, which does the following:
  - rewrite `mult a (Suc b)` to its definition `a + mult a b`,
  - use the assumption (which is specified to hold for any b) to rewrite `mult a b` to `a * b`,
  - rewrite `a * (Suc b)` to `a + a * b` using an internal definition of multiplication of natural numbers,
  - prove the resulting sub-goal `a + a * b = a + a * b` by reflection.
- the lemma is added to the theorem with `done`, which can only be done when no sub-goals are left over.

```isabelle
lemma "mult a b = a * b"
  apply (induction b)
  apply simp
  apply simp
  done
```

Listing 7: Lemma for `mult` Function

Moreover, Isabelle allows user to create custom datatypes to be used in functions and lemmas using the keyword `datatype`. These datatypes can have any number of uniquely-named constructors which can consist of any number of values of different types. It is also possible for datatypes to contain generic types, which can be instantiated with some type whenever they are used. Examples of a custom natural number and list datatype are shown in Listing 8. Here, `custom_nat` is defined as either `custom_zero`, representing zero, or `custom_succ custom_nat`, representing the successor of another `custom_nat`. The value shown below it represents the natural number `2`. Furtermore, `custom_list` uses a generic type `'a`, which is defined as either `custom_nil`, representing an empty list, or `custom_cons`, representing a value of `'a` prepended to another `custom_list` instantiated with the same generic type. The value shown below it represents the list with `[0, 1]`.

```isabelle
datatype custom_nat = custom_zero | custom_succ "custom_nat"
value "custom_succ (custom_succ custom_zero)"

datatype 'a custom_list = custom_nil | custom_cons "'a" "'a custom_list"
value "custom_cons custom_zero (custom_cons (custom_succ custom_zero) custom_nil)"
```

Listing 8: Isabelle Datatype Examples

As previously mentioned, Isabelle provides different reasoning tools to simplify, break up, and prove sub-goals. One such example is the Simplifier, as invoked in Listing 7, which supports higher-order rewriting. Additional tools include the classical reasoner, classical tableau prover, Metis prover, and Sledghammer. Users can use such tools, but also directly apply rules to rewrite or prove goals if their conditions for application are met. For example, the rule `refl` can prove sub-goals by reflection if the goal is of the form `x = x` for some `x`.

Moreover, users can define their own proof methods using Eisbach, Isabelle's built-in proof method language. With Eisbach, invoked with the `method` keyword, it is possible to combine different rules and reasoning tools for specific use-cases. Such user-defined methods can be applied the same way as other pre-defined proof methods, using the `apply` keyword followed by the specified method name. For example, the rule `asm_rl` can be applied, which only succeeds for sub-goals of a specific form, followed by another proof method capable of solving or simplifying that specific sub-goal. It also features built-in sub-goal pattern matching for this end, which is more powerful as it then allows using variables from matched sub-goal for proof methods.

The first technique to apply certain proof-methods for specific subgoals is used in Listing 9, where a similar proof as Listing 7 is given. Here, the method `example_base` can only be used for the base step of the induction proof, as applying `rule asm_rl[of "mult _ 0 = _"]` only succeeds when a sub-goal of that form is present. Similarly, the method `example_inductive` can only be used for the inductive step, as it needs a sub-goal where

the second parameter of `mult` is a successor of another natural number. Trying to apply the methods to other sub-goals fails.

```isabelle
method example_base = rule asm_rl[of "mult _ 0 = _"], simp
method example_inductive = rule asm_rl[of "mult _ (Suc _) = _"], simp

lemma "mult a b = a * b"
  apply (induction b)
  apply example_base
  apply example_inductive
  done
```

Listing 9: Custom Proof Methods in `mult` Lemma

3 Preliminary Results (WIP)

This section covers the preliminary results from work already done. Specifically, it details how `SG1` and `SG2` have already been achieved in Section 3.1 and Section 3.2 respectively, and how `SG3` is being tackled in Section 3.3.

3.1 LLVM-IR in Isabelle

A subset of LLVM-IR's structure and semantics have been defined in Isabelle. The chosen subset covers basic language features that allow simple programs, such as the `mult` function from Listing 4, to be described and executed. Specifically, the structure of some instructions, basic blocks, functions, and programs have been defined, as well as their semantics.

The structure has been defined using datatypes in the form of its AST. For example, a datatype `llvm_instruction` has been defined which can be instantiated as any of the supported instructions with their specific options according to their definition in LLVM's language reference manual. Its definition is shown in Listing 10. Such definitions have been made for types, typed values, identifiers, value references (either a direct value, or value from a register), alignment specifications, add wrap options, compare conditions, phi instructions, regular instructions, terminator instructions, basic blocks, block return values, functions, and programs.

```isabelle
datatype llvm_instruction = alloca llvm_register_name llvm_type "llvm_align option"
                          | store llvm_type llvm_value_ref llvm_pointer "llvm_align option"
                          | load llvm_register_name llvm_type llvm_pointer "llvm_align option"
                          | add llvm_register_name llvm_add_wrap llvm_type llvm_value_ref llvm_value_ref
                          | icmp llvm_register_name llvm_same_sign llvm_compare_condition llvm_type llvm_value_ref llvm_value_ref
```

Listing 10: Datatype Definition for LLVM-IR Instructions

On top of this representation of the AST, the semantics are defined using functions that form an interpreter. At the instruction level lies a function that takes in an execution state, a triple of register values, stack memory, and heap memory, as well as an `llvm_instruction`, and produces a new state result. This result can be successful, in which case it contains the new state, or be erroneous if some execution requirement was not met, for example when trying access a value in a register that has not been assigned. Using this function, a basic block executor is defined that executes its internal instructions in-order, taking in an execution state and basic block, and producing a result tuple of a new state and block return value. Its definition is shown in Listing 11. For this, a monad is defined for the `result` type, so errors automatically short-circuit execution, and successful execution flows naturally. Block return values contain information on whether to branch to a different basic block, or return from the function with a value. Finally, an executor for functions is defined that uses the basic block executor to execute its basic blocks, using the block return values to properly handle control-flow.

```isabelle
fun execute_block :: "llvm_label option ⇒ llvm_instruction_block ⇒ state ⇒ (state * llvm_block_return) result" where
  "execute_block previous ((phi name type values)#ps, is, t) s = do {
    s' ← execute_phi previous name values s;
    execute_block previous (ps, is, t) s'
  }"
| "execute_block previous ([], i#is, t) s = do {
    s' ← execute_instruction i s;
    execute_block previous ([], is, t) s'
  }"
| "execute_block previous (_, _, br_i1 v l1 l2) s = do {
    val ← get_register s v;
    label ← (case val of (vi1 b) ⇒ ok (if b then l1 else l2) | _ ⇒ err incompatible_types);
    return (s, branch_label label)
  }"
| "execute_block previous (_, _, br_label l) s = return (s, branch_label l)"
| "execute_block previous (_, _, ret t v) s = do {
    res ← get_register s v;
    return (s, return_value res)
  }"
```

Listing 11: Basic Block Execution Function

Together, the defined structure and semantics can be used to execute functions made up of the supported instructions.

3.2 Verification Infrastructure

- on top of the structure and semantics, weakest pre-condition infrastucture has been defined
- using these weakest pre-conditions, single instructions can be symbolically executed, giving the user proof-goals for their prerequisites to execute without errors and the resulting execution state, using proof-friendly abstractions
- also, entire instruction blocks can be executed symbolically, giving the user proof-goals for successful execution of the entire block and the final execution state, using the same abstractions

3.3 Verification Tools

- hoare triple definition to invoke weakest pre-condition infrastructure more intuitively

- annotated function datatype to specify pre-conditions for basic blocks in a function, as well as post-conditions for the function
- `verify` function that takes an annotated function and produces a boolean if it can be v
- Eisbach methods to automatically perform verification of sub-goals (along with debug variants), allowing the user to only have to deal with the final goals, or failing sub-goals (e.g. due to missing pre-conditions for basic blocks)
- TODO is ensuring this functionality works properly, handle edge-cases more elegantly, and clean it up

This section will cover the following information:

- `SG1` has been achieved:
- `SG2` has been achieved:
- `SG3` has been mostly achieved:
- `SG4` has not been achieved yet:
  - only a few simple programs have been put through the tool, too simple to say much about its true usefulness

4 Related Work (WIP)

This section will cover, among others, the following work:

- Framework for VCGs using theorem provers [1]
- Exporting from Isabelle to LLVM[2]
- Deductive verification backend[3]
- Write program in Dafny, verify, and then compile to your preferred language[4]

Deductive Verifiers:
- Verifying x86 binaries[5]
- RESOLVE[6]
- Whiley[7]
- Frama-C[8]
- KIV[9]
- OpenJML[10]
- VeriFast[11]
- VerCors[12]
  - WIP LLVM frontend[13]

...

5 Planning

Much of the minimum viable product has already been implemented without a clear planning. The remaining work will be implemented according to the GANTT chart shown in Figure 2. Furthermore, it shows the extra features to be implemented on top of the MVP, along with rough estimates on how long they will take to complete and when they might

be implemented. In addition to that, it shows roughly when work on the tool itself will conclude, and the accumulated work and research will be gathered and written into the final thesis document. The current aimed-for date to complete the thesis is the 19th of June, which can be extended if necessary.

5.1 Risk Mitigation

With any project, especially of the size of a thesis, risks are present that might endanger its feasibility and completion. For the completion of this project, running out of time before implementing all proposed features is the biggest risk due to uncertain development time and complexity. For this reason, a minimum viable product was defined (`SG1-4`) which represents the core goal of the research: creating a deductive verifier for a subset of LLVM-IR in Isabelle. When this core goal is achieved, additional features (`EG1-4`) can be implemented that are not strictly necessary for the functionality of the tool, but do extend its usefulness. Given that a large part of the MVP has already been implemented, there is only a low risk that it cannot be completed before the deadline. The extension goals have been formulated in an open-ended way so their scope can scale to the time left for their implementation, or some can be left out entirely if others are deemed more important. Currently, `EG2` and `EG3` are deemed most important for the usefulness of the final tool, so the initial time after completing `SG1-4` will be spent on them.

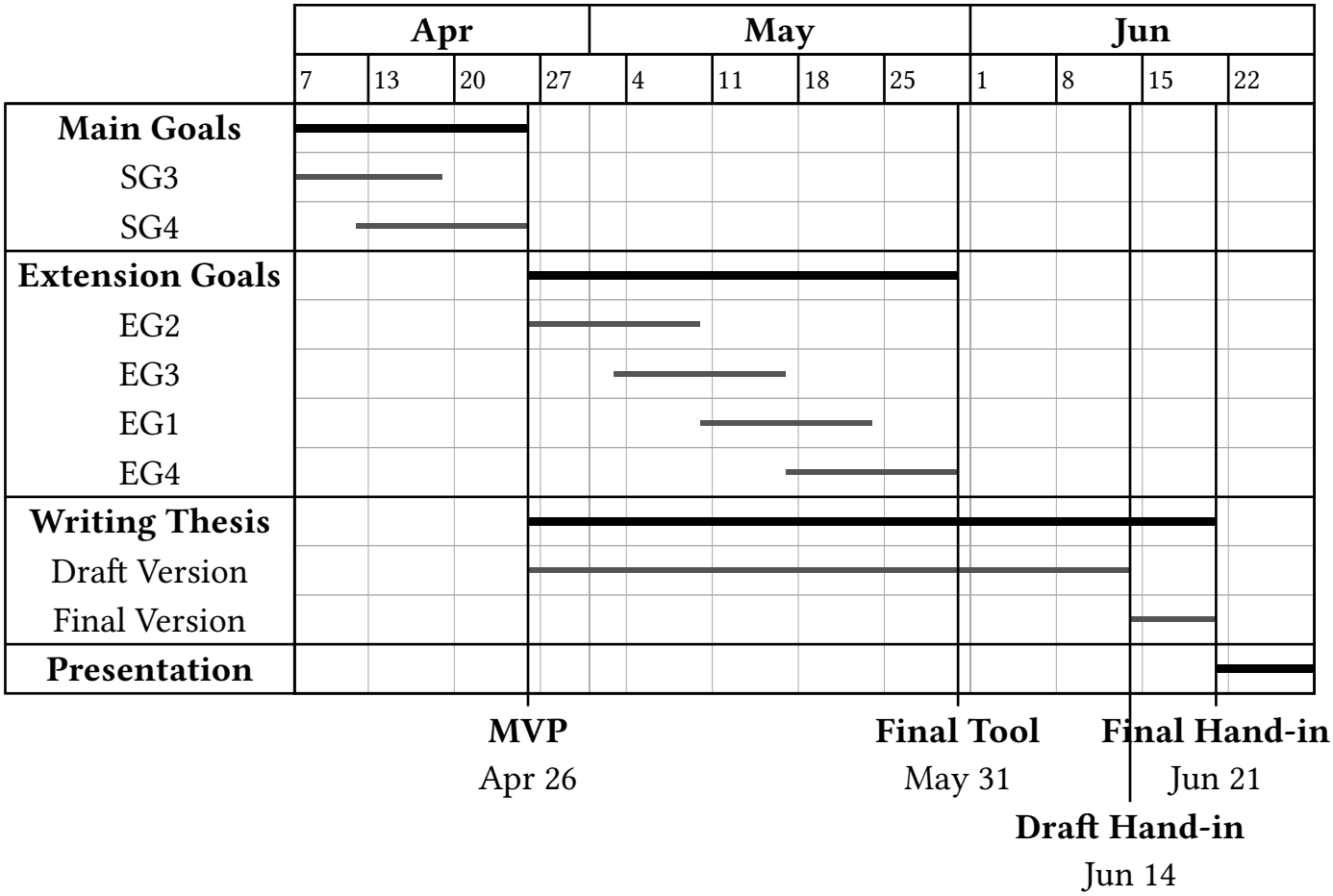

Figure 2: GANTT Chart of Project Planning

Bibliography

[1] J. Matthews, J. S. Moore, S. Ray, and D. Vroon, "Verification Condition Generation Via Theorem Proving," in *Logic for Programming, Artificial Intelligence, and Reasoning*, Springer Berlin Heidelberg, 2006, pp. 362–376. doi: 10.1007/11916277_25.

[2] P. Lammich, "Generating Verified LLVM from Isabelle/HOL," Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 22:1–22:19. doi: 10.4230/LIPICS.ITP.2019.22.

[3] P. Müller, M. Schwerhoff, and A. J. Summers, "Viper: A Verification Infrastructure for Permission-Based Reasoning," in *Verification, Model Checking, and Abstract Interpretation*, Springer Berlin Heidelberg, 2015, pp. 41–62. doi: 10.1007/978-3-662-49122-5_2.

[4] K. R. M. Leino, "Dafny: An Automatic Program Verifier for Functional Correctness," in *Logic for Programming, Artificial Intelligence, and Reasoning*, Springer Berlin Heidelberg, 2010, pp. 348–370. doi: 10.1007/978-3-642-17511-4_20.

[5] J. A. Bockenek, F. Verbeek, P. Lammich, and B. Ravindran, "Formal Verification of Memory Preservation of x86-64 Binaries," in *38th International Conference on Computer Safety, Reliability and Security*, Sept. 2019.

[6] M. Sitaraman and B. W. Weide, "A Synopsis of Twenty Five Years of RESOLVE PhD Research Efforts," *ACM SIGSOFT Software Engineering Notes*, vol. 43, no. 3, p. 17, Dec. 2018, doi: 10.1145/3229783.3229794.

[7] D. J. Pearce, M. Utting, and L. Groves, "An Introduction to Software Verification with Whiley," in *Engineering Trustworthy Software Systems*, Springer International Publishing, 2019, pp. 1–37. doi: 10.1007/978-3-030-17601-3_1.

[8] A. Blanchard, F. Loulergue, and N. Kosmatov, "Towards Full Proof Automation in Frama-C Using Auto-active Verification," in *NASA Formal Methods*, Springer International Publishing, 2019, pp. 88–105. doi: 10.1007/978-3-030-20652-9_6.

[9] G. Ernst, J. Pfähler, G. Schellhorn, D. Haneberg, and W. Reif, "KIV: overview and VerifyThis competition," *International Journal on Software Tools for Technology Transfer*, vol. 17, no. 6, pp. 677–694, Apr. 2014, doi: 10.1007/s10009-014-0308-3.

[10] D. R. Cok, "OpenJML: JML for Java 7 by Extending OpenJDK," in *NASA Formal Methods*, Springer Berlin Heidelberg, 2011, pp. 472–479. doi: 10.1007/978-3-642-20398-5_35.

[11] B. Jacobs, J. Smans, P. Philippaerts, F. Vogels, W. Penninckx, and F. Piessens, "VeriFast: A Powerful, Sound, Predictable, Fast Verifier for C and Java," in *NASA Formal Methods*, Springer Berlin Heidelberg, 2011, pp. 41–55. doi: 10.1007/978-3-642-20398-5_4.

[12] S. Blom, S. Darabi, M. Huisman, and W. Oortwijn, “The VerCors Tool Set: Verification of Parallel and Concurrent Software,” in *Integrated Formal Methods*, Springer International Publishing, 2017, pp. 102–110. doi: 10.1007/978-3-319-66845-1_7.

[13] D. van Oorschot, M. Huisman, and Ö. Şakar, “First Steps towards Deductive Verification of LLVM IR,” in *Fundamental Approaches to Software Engineering*, Springer Nature Switzerland, 2024, pp. 290–303. doi: 10.1007/978-3-031-57259-3_15.